

Hackathon Problem Statement

Problem Statement: Intelligent Mock Interview Agent

Objective

Design an **Agentic AI-powered mock interview platform** (APP / web site / cli tools / Raw code scripts) that uses a candidate's **resume** to infer the **most likely job roles** they can realistically get into, then simulates role-appropriate interview scenarios.

- Candidate Resume - PDF/Text input
- Live video + audio feed

The system should **interact with the candidate**, dynamically generate questions, and evaluate both **technical competency** and **behavioural attributes** such as confidence and communication.

Core System Architecture (Agent-Driven)

1. Context Understanding Module

- Parse resume extract candidate skills, experience, projects, seniority signals, and domain exposure.
 - Infer likely target roles (e.g., Backend, Frontend, Data Analyst, DevOps, QA, Product, etc.) based on resume evidence and market patterns.
 - Generate interview focus areas per inferred role:
 - Core skills to validate
 - Missing/weak areas to probe (based on resume signals)
 - Project deep-dives and impact verification
-

2. Interview Orchestrator Agent

Acts as the **central decision-maker**.

Responsibilities:

- Generate interview questions dynamically
- Adapt questioning strategy based on:
 - Candidate responses
 - Confidence levels
 - Knowledge depth

Example Behaviour:

“Candidate struggled with system design → shift to fundamentals before increasing difficulty”

3. 🎤 Audio Intelligence (Speech + Emotion)

- Speech-to-text transcription
- Analyse:
 - Tone
 - Pitch variation
 - Hesitation / pauses

👉 Output:

- Confidence score
 - Communication clarity score
-

4. 🖥️ Visual Intelligence (Computer Vision) {Prefer using pretrained CV model over LLMS}

- Analyse:
 - Eye contact
 - Facial expressions
 - Posture

👉 Output:

- Engagement score
 - Stress / nervousness indicators
-

5. 🌱 Technical Evaluation Engine { Focus on System Architecture over LLM correctness}

- Evaluate answers using:
 - LLM-based semantic scoring
 - Keyword/intent matching

👉 Output:

- Correctness score
 - Depth of knowledge
-

6. ⚖️ Feedback & Coaching Agent

- Provides structured feedback:
 - Technical gaps
 - Communication improvements
 - Behavioural insights

Example:

“Strong conceptual understanding, but low vocal confidence—recommend slowing down and structuring responses.”

Key Features (Must-Have)

- Dynamic question generation (not static list)
- Real-time adaptation based on candidate performance
- Multimodal scoring:
 - Technical
 - Communication
 - Confidence
- Explainable feedback
- Any extra features in a platform would be considered as bonus with great weightage.
- **Resume-based job recommendations:** a section where users upload their resume and the platform suggests the best-fit jobs to apply for by crawling/aggregating currently available and suitable postings, ranking them by match score and highlighting why each role fits.

Evaluation Criteria

1. Agent Intelligence

- Does the system adapt or just follow a script?

2. Multimodal Integration

- How well are audio, video, and text signals combined?

3. Feedback Quality

- Actionable vs generic insights

4. User Experience

- Natural interaction flow

Practical Constraints (Important for Hackathon)

- Pre-recorded video/audio allowed if real-time is hard
- Simplified CV models acceptable (heuristics-based scoring)
- Focus more on **agent logic than model perfection**

Optional Enhancement (To Make It Stand Out Further)

You can introduce:

- If candidate confidence drops:
 - Agent switches to easier questions
 - Provides encouragement
 - Gradually ramps difficulty
- Role-specific interviews (e.g., Backend, ML, Embedded)
- Coding round simulation (optional)
- Real-time hints / coaching mode
- Historical performance tracking

Deliverables

Demo

A short demo video showing end-to-end platform usage, including:

- Input upload: resume (PDF/text) and how it is parsed to infer likely job roles.
- Job discovery: resume-based job recommendations (crawled/aggregated postings), match scoring, and suggested applications list.
- Live (or prerecorded) interview flow: question generation, adaptation, and scoring.
- Sample feedback report/coaching output (technical + communication/confidence).
- Any optional enhancements implemented.

Code Base

Complete codebase with clear documentation to build, configure, and run the solution (local or cloud), including:

- README with prerequisites, setup steps, and one-command run instructions.
- Project structure overview and how the agents/modules interact.
- Sample inputs (example JD/resume) and expected outputs (scores/feedback) for quick evaluation.

Documentation

A 1–2-page architecture document describing the end-to-end design, including:

- Problem summary and user journey.
- High-level architecture (modules/agents, data flow, and interfaces).
- Key design choices and trade-offs (latency, scalability, model selection).
- Scoring approach (technical + audio/video signals) and how outputs are aggregated.
- Limitations, assumptions, and next steps.